

Introduction to Big Data Analytics

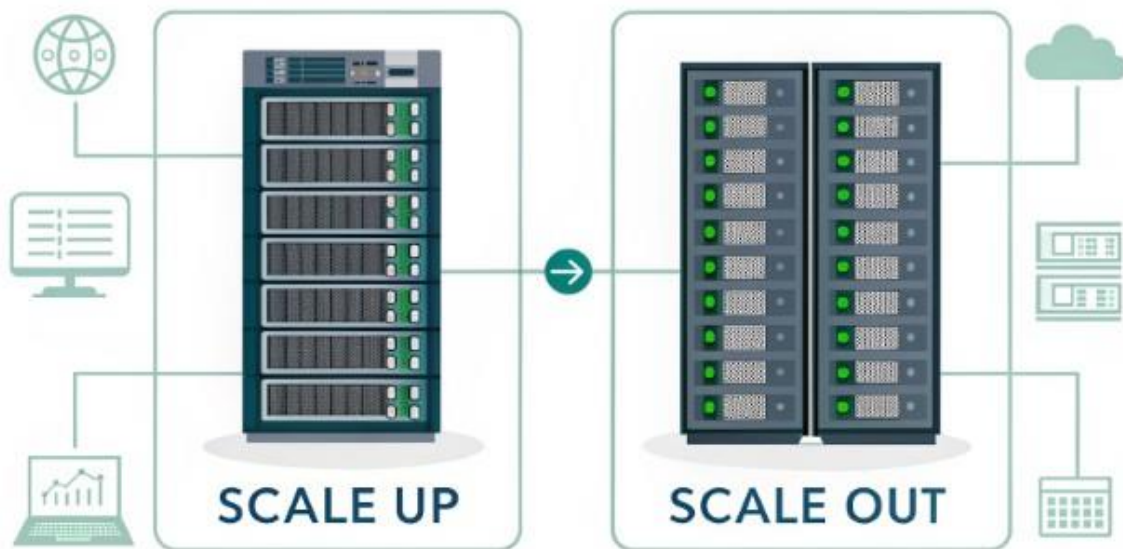
In modern computing environments, Big Data refers to datasets whose size, complexity, and rate of generation exceed the storage capacity, processing capability, and algorithmic limits of traditional relational database management systems (RDBMS). Big Data is typically characterized by the 5 Vs:

1. Volume: The massive scale of data generated every second (terabytes, petabytes, and exabytes) from sources like IoT sensors, transactional logs, and social media networks.
2. Velocity: The rapid speed at which new data is generated, transmitted, and required to be processed in real-time or near-real-time.
3. Variety: The diversity of data formats, including:
 - Structured: Relational tables, CSV files.
 - Semi-Structured: JSON, XML, key-value stores.
 - Unstructured: Images, raw audio/video files, plain text logs.
4. Veracity: The trustworthiness, noise level, quality, and accuracy of the incoming data streams.
5. Value: The actionable business insights, statistical trends, and predictive intelligence derived from analyzing raw data.

2. Monolithic vs. Distributed Computing Architectures

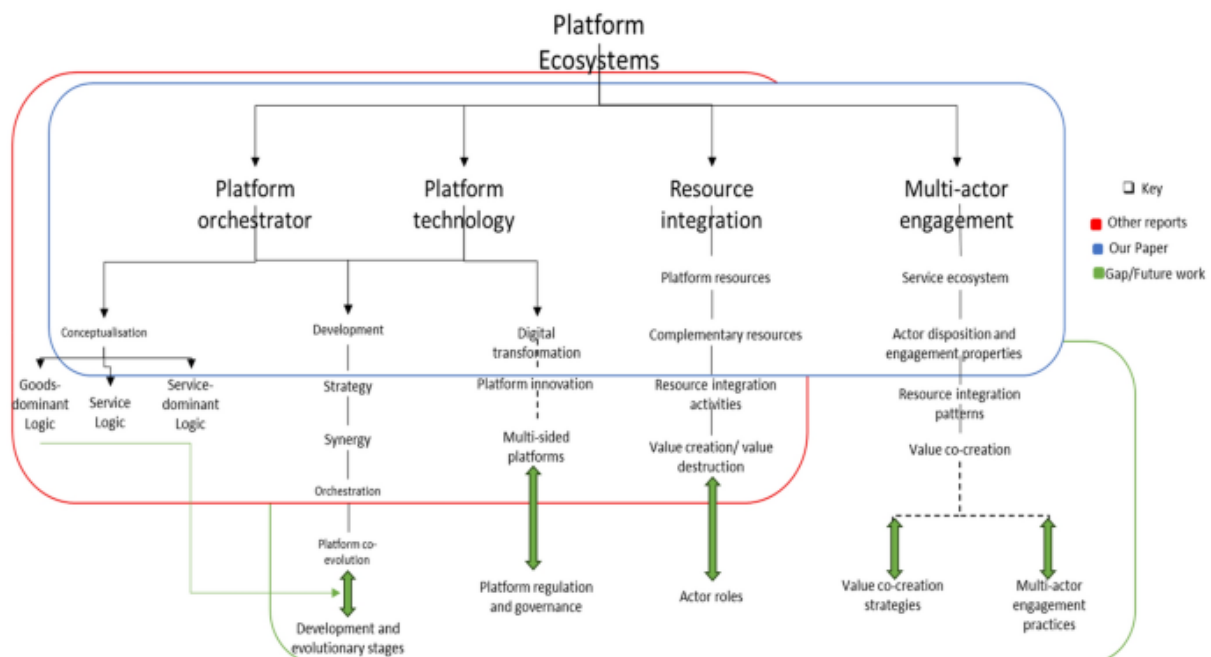
Traditional database systems rely on Vertical Scaling (Scale-Up)—adding more RAM, CPU cores, or storage drives to a single server. However, vertical scaling faces hardware physical limits and exponential costs.

Big Data Analytics relies on Horizontal Scaling (Scale-Out) using Distributed Systems. Instead of relying on a single supercomputer, a cluster of commodity (standard, lower-cost) hardware nodes work in unison.



3. Core Ecosystems & Conceptual Frameworks

The Big Data pipeline, spanning data ingestion, distributed storage, batch computation, interactive querying, real-time streaming, and exploratory data analysis.



A. Local Scientific Stack & Visualization

Before scaling out to distributed clusters, data scientists analyze data subsets locally.

- NumPy: Provides memory-efficient multi-dimensional arrays and linear algebra mathematical functions.
- Pandas: Offers tabular data structures (DataFrames) for dataset manipulation, filtering, and summary statistics.
- Matplotlib & Seaborn: Generate static visual plots (line charts, histograms, scatter plots, bar charts) to detect data distributions, anomalies, and correlations.

B. Distributed Storage & Batch Computation: Apache Hadoop Ecosystem

Apache Hadoop addresses the challenge of storing and processing massive files across a cluster.

- Hadoop Distributed File System (HDFS): Breaks large files into fixed-size blocks (e.g., 128MB) and distributes them across a cluster of nodes. HDFS uses a Master/Slave architecture:
 - NameNode (Master): Manages file system namespace, metadata, and block locations.
 - DataNode (Slave): Stores actual data blocks and handles block read/write operations.
- MapReduce Programming Paradigm: A distributed computation model operating in two main phases:
 1. Map Phase: Reads input data blocks concurrently, parses raw records, and outputs intermediate key-value pairs (K1, V1) -> List(K2, V2).
 2. Shuffle & Sort Phase: Groups all intermediate values associated with the same intermediate key together.
 3. Reduce Phase: Aggregates, filters, or summarizes intermediate grouped values to produce the final output (K2, List(V2)) -> List(K3, V3).

C. NoSQL Databases & Data Warehousing

MongoDB (NoSQL): A document-oriented database that stores semi-structured data in JSON/BSON document formats. It supports dynamic schemas, embedded arrays, and aggregation pipelines without requiring strict relational joins.

- Apache Pig: A high-level data-flow platform that executes scripts written in Pig Latin. It abstracts complex Java MapReduce programs into relational algebra operations (FILTER, JOIN, GROUP) and supports User Defined Functions (UDFs) in Java for custom processing.

- Apache Hive: A distributed data warehouse system built on top of Hadoop. It provides an SQL-like query language (HiveQL) that translates declarative SQL queries into lower-level MapReduce or Tez jobs.

D. In-Memory Computing & Real-Time Streaming

- Apache Spark: An in-memory distributed data processing engine that addresses Hadoop MapReduce's disk I/O bottlenecks.
 - Resilient Distributed Datasets (RDDs): Fault-tolerant, immutable collections of objects partitioned across cluster nodes.
 - Spark DataFrames: Strongly-typed, columnar optimized data structures powered by Spark's Catalyst Optimizer.
- Spark Structured Streaming: A stream processing engine built on the Spark SQL engine, treating live data streams as unbounded, continuously appended tables.
- Apache Kafka: A distributed event-streaming platform operating on a publish-subscribe model:
 - Producers: Publish event messages to specific Kafka Topics.
 - Brokers: Cluster servers that persist and replicate topic partitions.
 - Consumers: Subscribe to topics and pull data streams sequentially for real-time processing.

4. Big Data Technology Comparison Matrix

Framework / Tool	Layer / Category	Primary Data Model	Key Strength	Ideal Use Case	Manual Reference
Python Stack	Analytics & EDA	Memory Arrays / DataFrames	Rapid prototyping, rich plotting	Small/Medium data visualization	Labs 1, 2
Hadoop HDFS	Distributed Storage	Block File System	High throughput, disk fault tolerance	Storing raw unstructured/log files	Labs 1, 3

Framework / Tool	Layer / Category	Primary Data Model	Key Strength	Ideal Use Case	Manual Reference
Hadoop MapReduce	Batch Processing	Disk Key-Value Pairs	Stable batch processing on commodity nodes	Heavy nightly batch transformations	Lab 4
Apache Pig	Data-Flow Scripting	Tuple / Bag Schema	High-level data transformation pipelines	Custom ETL pipelines via Java UDFs	Lab 5
MongoDB	NoSQL Database	BSON / Documents	Schema-free, nested arrays, fast reads	Flexible catalogs, session stores	Lab 6
Apache Hive	Data Warehousing	Schema-on-Read Tables	SQL-interface over distributed storage	Ad-hoc queries on historical logs	Lab 7
Apache Spark	In-Memory Processing	RDDs / DataFrames	In-memory execution, low latency	Iterative machine learning, ETL	Lab 8
Spark Streaming	Stream Processing	Unbounded DataFrames	Micro-batching with end-to-end guarantees	Rolling window aggregations	Lab 9
Apache Kafka	Event Streaming	Partitioned Log Streams	High-throughput, low-latency pub-sub	Ingestion pipeline for live logs/events	Lab 10

Lab 1: Installation, Configuration, and Verification

Objective

Install and verify the core Python data science stack (NumPy, Pandas, Matplotlib) along with single-node Hadoop (HDFS).

Step-by-Step Execution

Step 1: Install Python & Libraries

Bash

```
# Update package list and install Python 3 & pip
sudo apt update
sudo apt install python3 python3-pip -y
```

```
# Install core scientific and visualization packages
pip3 install numpy pandas matplotlib seaborn
```

Step 2: Set Up Java (Prerequisite for Hadoop)

Bash

```
sudo apt install openjdk-8-jdk -y
java -version
```

Step 3: Install and Configure Single-Node Hadoop

Bash

```
# Download and extract Hadoop 3.3.6
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzvf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
```

```
# Set Environment Variables in ~/.bashrc
cat << 'EOF' >> ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
EOF
```

```
source ~/.bashrc
```

Verification Program (lab1_setup.py)

Python

```
import numpy as np
import pandas as pd
```

NumPy Verification

```
arr = np.array([10, 20, 30, 40, 50])
print(f"NumPy Mean: {np.mean(arr)}")
```

Pandas Verification

```
df = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [85, 92, 78]
})
print("\nPandas DataFrame:")
print(df)
```

Command & Expected Output

Bash

```
python3 lab1_setup.py
```

Plaintext

NumPy Mean: 30.0

Pandas DataFrame:

	ID	Name	Score
0	101	Alice	85
1	102	Bob	92
2	103	Charlie	78

Lab 2: Data Visualization in Python

Objective

Visualize datasets using line plots, bar charts, histograms, and scatter plots using Matplotlib and Seaborn.

Program Code (lab2_visualization.py)

Python

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Generate dummy dataset
np.random.seed(42)
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [12000, 15000, 18000, 14000, 22000, 25000]
df = pd.DataFrame({'Month': months, 'Sales': sales})

# Create a multi-panel plot layout
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 1. Line Plot
axes[0, 0].plot(df['Month'], df['Sales'], marker='o', color='b', linewidth=2)
axes[0, 0].set_title('Monthly Sales Trend (Line Plot)')

# 2. Bar Chart
axes[0, 1].bar(df['Month'], df['Sales'], color='skyblue')
axes[0, 1].set_title('Monthly Revenue Comparison (Bar Chart)')

# 3. Histogram
data = np.random.normal(loc=100, scale=15, size=500)
axes[1, 0].hist(data, bins=20, color='lightgreen', edgecolor='black')
axes[1, 0].set_title('Distribution Analysis (Histogram)')

# 4. Scatter Plot
x = np.random.rand(50) * 100
y = x * 2 + np.random.randn(50) * 10
```

```
axes[1, 1].scatter(x, y, color='purple', alpha=0.7)
axes[1, 1].set_title('Scatter Relationship Plot')

plt.tight_layout()
plt.savefig('visualization_output.png')
print("Plot successfully saved as 'visualization_output.png'")
```

Expected Console Output

Plaintext
Plot successfully saved as 'visualization_output.png'

Lab 3: Basic HDFS Commands & File Operations

Objective

Demonstrate reading, writing, copying, and directory manipulation in the Hadoop Distributed File System (HDFS).

Execution Steps & Commands

Bash

1. Start NameNode and DataNode daemons
start-dfs.sh

2. Create a directory structure in HDFS
hdfs dfs -mkdir -p /user/hadoop/lab_data

3. Create a local sample file
echo "Hadoop HDFS File System Demonstration File" > sample_local.txt

4. Write (Upload) file from local machine to HDFS
hdfs dfs -put sample_local.txt /user/hadoop/lab_data/

5. List HDFS directory contents
hdfs dfs -ls /user/hadoop/lab_data/

6. Read file contents directly from HDFS
hdfs dfs -cat /user/hadoop/lab_data/sample_local.txt

7. Copy file inside HDFS
hdfs dfs -cp /user/hadoop/lab_data/sample_local.txt
/user/hadoop/lab_data/backup.txt

8. Read (Download) file back to local file system
hdfs dfs -get /user/hadoop/lab_data/backup.txt ./downloaded_backup.txt

Expected Command Line Outputs

Plaintext

```
$ hdfs dfs -ls /user/hadoop/lab_data/
```

Found 1 items

```
-rw-r--r-- 1 hadoop supergroup      43 2026-08-04 14:00  
/user/hadoop/lab_data/sample_local.txt
```

```
$ hdfs dfs -cat /user/hadoop/lab_data/sample_local.txt
```

Hadoop HDFS File System Demonstration File

Lab 4: MapReduce Applications

Lab 4a: Word Count Application (Single Node)

Java MapReduce Code (WordCount.java)

Java

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context) throws IOException,
            InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken().toLowerCase().replaceAll("[^a-zA-Z]", ""));
                if (word.getLength() > 0) {
                    context.write(word, one);
                }
            }
        }
    }

    public static class IntSumReducer
```

```

    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
    throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution & Execution Commands

Bash

Set HADOOP CLASSPATH and Compile

```
export HADOOP_CLASSPATH=$(HADOOP_HOME/bin/hadoop classpath)
```

```
javac -classpath $HADOOP_CLASSPATH -d . WordCount.java
```

```
jar cvf wordcount.jar WordCount*.class
```

Prepare Input Data

```
echo "hadoop is big data framework. mapreduce processes big data." > input.txt
```

```
hdfs dfs -mkdir -p /wc/input
```

```
hdfs dfs -put input.txt /wc/input/
```

```
# Run MapReduce Job
```

```
hadoop jar wordcount.jar WordCount /wc/input /wc/output
```

Expected Output (hdfs dfs -cat /wc/output/part-r-00000)

Plaintext

big 2

data 2

framework 1

hadoop 1

is 1

mapreduce 1

processes 1

Lab 4b: Weather Dataset Analysis (Max Temperature)

Mapper (WeatherMapper.java)

Java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
```

```
public class WeatherMapper extends Mapper<LongWritable, Text, Text,
IntWritable> {
    @Override
    public void map(LongWritable key, Text value, Context context) throws
IOException, InterruptedException {
        String line = value.toString();
        if (line.length() >= 15) {
            String year = line.substring(0, 4);
            int temp = Integer.parseInt(line.substring(5, 10).trim());
            context.write(new Text(year), new IntWritable(temp));
        }
    }
}
```

Reducer (WeatherReducer.java)

Java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WeatherReducer extends Reducer<Text, IntWritable, Text,
IntWritable> {
    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException, InterruptedException {
        int maxTemp = Integer.MIN_VALUE;
        for (IntWritable val : values) {
            maxTemp = Math.max(maxTemp, val.get());
        }
        context.write(key, new IntWritable(maxTemp));
    }
}
```

Execution

Bash

```
# Input file contents (sample_weather.txt):
# 2023 0035
# 2023 0042
# 2024 0038
# 2024 0045
```

```
hdfs dfs -mkdir -p /weather/input
hdfs dfs -put sample_weather.txt /weather/input/
hadoop jar weather.jar WeatherDriver /weather/input /weather/output
```

Output

Plaintext

```
2023 42
2024 45
```

Lab 4c: Real-World Case Study Overview

Industry Vertical	MapReduce Job Profile	Input Data Stream	Output Business Value
E-Commerce	Recommendation	User	Product Co-
	Engines & Association	Clickstream	occurrence
	Rules	Logs	Counts
Telecommunications	Call Detail Record	Network Logs	Peak Usage &
	(CDR) Aggregation		Network Load Maps
Finance	Batch Fraud Analysis	Nightly Transaction Batches	Anomaly Detection Flags

Lab 5: Apache Pig - User Defined Functions (UDF)

Objective

Write an Eval UDF in Java to filter unwanted data records in Apache Pig scripts.

Java UDF Code (CleanDataUDF.java)

```
Java
package com.lab.pig;

import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;

public class CleanDataUDF extends EvalFunc<String> {
    @Override
    public String exec(Tuple input) throws IOException {
        if (input == null || input.size() == 0 || input.get(0) == null) {
            return null;
        }
        String str = (String) input.get(0);
        // Clean input string: remove special symbols and convert to UPPERCASE
        String cleaned = str.replaceAll("[^a-zA-Z0-9 ]", "").trim().toUpperCase();
        return cleaned.isEmpty() ? null : cleaned;
    }
}
```

Pig Script (filter_script.pig)

Code snippet

```
-- Register custom JAR file
REGISTER /path/to/CleanDataUDF.jar;

-- Load Raw Data
raw_data = LOAD '/user/hadoop/pig_input/raw_logs.csv' USING PigStorage(',')
AS (id:int, message:chararray);

-- Clean data using User Defined Function
```

```
cleaned_tuples = FOREACH raw_data GENERATE id,  
com.lab.pig.CleanDataUDF(message) AS clean_msg;  
  
-- Filter out nulls/unwanted records  
filtered_data = FILTER cleaned_tuples BY clean_msg IS NOT NULL;  
  
-- Store Results  
STORE filtered_data INTO '/user/hadoop/pig_output' USING PigStorage(',');
```

Output (/user/hadoop/pig_output/part-m-00000)

Plaintext

1,VALID USER SESSION ACTIVE

2,PURCHASE COMPLETED SUCCESSFULLY

Lab 6: NoSQL Operations using MongoDB

Objective

Perform CRUD operations and manipulate embedded arrays in MongoDB shell.

Step-by-Step Execution Script

JavaScript

// Connect and Create/Switch Database

use store_db;

// 1. CREATE (Insert Documents with Embedded Arrays)

```
db.products.insertMany([
  {
    _id: 1,
    item: "Laptop",
    price: 1200,
    tags: ["electronics", "computers", "tech"],
    ratings: [5, 4, 5]
  },
  {
    _id: 2,
    item: "Mouse",
    price: 25,
    tags: ["electronics", "accessories"],
    ratings: [3, 4]
  }
]);
```

// 2. READ (Find Query with Array Filter)

```
db.products.find({ tags: "tech" }).pretty();
```

// 3. UPDATE (Modify Array - Append new tag & update price)

```
db.products.updateOne(
  { _id: 1 },
  {
    $set: { price: 1150 },
    $push: { tags: "sale" }
  }
);
```

```
);
```

```
// 4. ARRAY AGGREGATION & UNWINDING
```

```
db.products.aggregate([  
  { $unwind: "$tags" },  
  { $group: { _id: "$tags", count: { $sum: 1 } } }  
]);
```

```
// 5. DELETE
```

```
db.products.deleteOne({ _id: 2 });
```

Output Snippet

JSON

```
{  
  "_id" : 1,  
  "item" : "Laptop",  
  "price" : 1150,  
  "tags" : [  
    "electronics",  
    "computers",  
    "tech",  
    "sale"  
  ],  
  "ratings" : [ 5, 4, 5 ]  
}
```

Lab 7: Hive & User Defined Functions

Lab 7a: HiveQL DDL/DML Operations

SQL

-- Create Managed Database & Table

```
CREATE DATABASE IF NOT EXISTS lab_hive;  
USE lab_hive;
```

```
CREATE TABLE IF NOT EXISTS employee (  
    emp_id INT,  
    name STRING,  
    dept STRING,  
    salary DOUBLE  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

-- Load Data into Hive Table

```
LOAD DATA LOCAL INPATH '/home/hadoop/emp_data.csv' INTO TABLE  
employee;
```

-- Complex HiveQL Query with Aggregation

```
SELECT dept, COUNT(*) as total_emp, AVG(salary) as avg_sal  
FROM employee  
GROUP BY dept  
HAVING avg_sal > 50000;
```

Lab 7b: Writing Hive Java UDF

Java

```
package com.lab.hive;
```

```
import org.apache.hadoop.hive.ql.exec.UDF;  
import org.apache.hadoop.io.Text;
```

```
public class MaskSalaryUDF extends UDF {  
    public Text evaluate(Text input) {  
        if (input == null) return null;
```

```
        return new Text("XXXX-CONFIDENTIAL");
    }
}
SQL
-- Registering and Running UDF in Hive
ADD JAR /path/to/MaskSalaryUDF.jar;
CREATE TEMPORARY FUNCTION mask_sal AS
'com.lab.hive.MaskSalaryUDF';

SELECT name, mask_sal(CAST(salary AS STRING)) FROM employee;
```

Output

Plaintext

Alice	XXXX-CONFIDENTIAL
Bob	XXXX-CONFIDENTIAL

Lab 8: Processing & Word Count on Apache Spark

Lab 8a & 8b: PySpark Processing Script (spark_lab.py)

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split, col, lower

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("PySpark Execution Lab") \
    .master("local[*]") \
    .getOrCreate()

# Section A: Large Dataset RDD Processing
raw_rdd = spark.sparkContext.parallelize(range(1, 1000000))
even_sum = raw_rdd.filter(lambda x: x % 2 == 0).reduce(lambda a, b: a + b)
print(f"Sum of Evens in 1 Million Integers: {even_sum}")

# Section B: Spark Dataframe Word Count
data_df = spark.createDataFrame([
    (1, "Apache Spark handles large scale data processing efficiently."),
    (2, "Spark streaming and Spark SQL are essential components.")
], ["id", "text"])

# Tokenize and count words
words_df = data_df.select(explode(split(lower(col("text"))), " ").alias("word"))
word_counts = words_df.groupBy("word").count().orderBy(col("count").desc())

print("\nWord Count Top Results:")
word_counts.show(5)

spark.stop()
```

Expected Execution Output

Plaintext

Sum of Evens in 1 Million Integers: 249999500000

Word Count Top Results:

```
+-----+-----+
| word|count|
+-----+-----+
| spark| 3|
| data| 1|
| handles| 1|
| large| 1|
| and| 1|
+-----+-----+
```

only showing top 5 rows

Lab 9: Spark Structured Streaming (Retail Dataset)

Objective

Read streaming retail sales data using Spark Structured Streaming and execute rolling aggregations.

Program Code (structured_streaming.py)

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import expr
from pyspark.sql.types import DoubleType, IntegerType, StringType, StructType
```

```
spark = SparkSession.builder \
    .appName("RetailStructuredStreaming") \
    .master("local[*]") \
    .getOrCreate()
```

Define schema for retail transactions

```
schema = StructType() \
    .add("InvoiceNo", StringType()) \
    .add("StockCode", StringType()) \
    .add("Quantity", IntegerType()) \
    .add("UnitPrice", DoubleType()) \
    .add("Country", StringType())
```

Read streaming JSON files placed in a directory

```
streaming_df = spark.readStream \
    .schema(schema) \
    .option("maxFilesPerTrigger", 1) \
    .json("/tmp/retail_stream_input")
```

Compute Total Spend

```
processed_df = streaming_df \
    .withColumn("TotalSpend", expr("Quantity * UnitPrice")) \
    .groupBy("Country") \
    .sum("TotalSpend")
```

Output to console sink

```
query = processed_df.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

# Process stream for 10 seconds and stop
query.awaitTermination(10)
```

Expected Output

Plaintext

Batch: 0

Country	sum(TotalSpend)
United Kingdom	1245.80
Germany	340.50
France	185.20

Lab 10: Event Streaming using Apache Kafka

Objective

Create a Kafka Producer to send stream records and a Kafka Consumer to process them.

Execution Steps

Step 1: Start Zookeeper and Kafka Server

Bash

```
# Start Zookeeper
```

```
bin/zookeeper-server-start.sh config/zookeeper.properties &
```

```
# Start Kafka Broker
```

```
bin/kafka-server-start.sh config/server.properties &
```

```
# Create Topic
```

```
bin/kafka-topics.sh --create --topic lab-events --bootstrap-server localhost:9092 --  
partitions 1 --replication-factor 1
```

Step 2: Python Kafka Producer (producer.py)

Python

```
import json
```

```
import time
```

```
from kafka import KafkaProducer
```

```
producer = KafkaProducer(
```

```
    bootstrap_servers=['localhost:9092'],
```

```
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
```

```
)
```

```
data_stream = [
```

```
    {"event_id": 1, "device": "sensor_A", "status": "NORMAL"},
```

```
    {"event_id": 2, "device": "sensor_B", "status": "WARNING"},
```

```
    {"event_id": 3, "device": "sensor_A", "status": "CRITICAL"}]
```

```
for record in data_stream:
```

```
producer.send('lab-events', record)
print(f"Produced: {record}")
time.sleep(1)
```

```
producer.flush()
```

Step 3: Python Kafka Consumer (consumer.py)

Python

```
import json
```

```
from kafka import KafkaConsumer
```

```
consumer = KafkaConsumer(
    'lab-events',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
```

```
print("Listening for incoming stream events...")
for message in consumer:
    event = message.value
    print(f"Received Event: {event['event_id']} | Device: {event['device']} | Status: {event['status']}")
```

Terminal Console Output

Producer Output

Plaintext

```
Produced: {'event_id': 1, 'device': 'sensor_A', 'status': 'NORMAL'}
Produced: {'event_id': 2, 'device': 'sensor_B', 'status': 'WARNING'}
Produced: {'event_id': 3, 'device': 'sensor_A', 'status': 'CRITICAL'}
```

Consumer Output

Plaintext

```
Listening for incoming stream events...
Received Event: 1 | Device: sensor_A | Status: NORMAL
Received Event: 2 | Device: sensor_B | Status: WARNING
Received Event: 3 | Device: sensor_A | Status: CRITICAL
```